

Assignment: Sync Specs to Digital.ai Agility

Goal: Export backlog from `features/` into [Digital.ai Agility](#) and keep Agility in sync when stories or Gherkin change. Specs remain the source of truth; Agility is the planning board (epics, stories, acceptance tests, sprints).

Related docs: [docs/agility-import/README.md](#) · [features/framework.md](#) (Merge checklist + Agility sync) · [features/README.md](#)

Learning outcomes

By the end of this assignment you will be able to:

1. Explain how Speckit maps to Agility (feature → epic, `US-N.n` → story, Gherkin → test).
2. Configure Agility credentials (API path) or prepare an Excel import workbook (CSV path).
3. Export or push backlog from `features/feature-N-*.md` into your Agility project.
4. Re-sync after a spec change without blindly duplicating assets (upsert / new-items-only).

Prerequisites

Tool	Notes
This Speckit repo	Opened in Cursor; features already present (Todo example or your own)
Node.js 24+ / npm	For export / push scripts
Digital.ai Agility access	Product Planning edit rights on a project
Access token (API path)	Agility → My Settings → Access Tokens
Excel (CSV path only)	Agility Excel import template (.xls)

How Speckit maps to Agility

```
features/feature-N-....md      → Portfolio Item (Epic)
  US-N.n user story            → Story
    ##### Scenario (Gherkin)  → Test (acceptance)
```

Speckit	Agility
# Feature: ... heading	Epic Name
User story US-N.n	Story (Reference like TS-F3-US3.1)
Gherkin scenario	Test under that story
Sprint / iteration	Assigned in Agility (not in the Markdown)

Feature files matching `features/feature-N-*.md` are **auto-discovered**. You do not edit a hard-coded file list when adding a feature.

Choose one path for this assignment (your instructor may require both or only one):

Path	Best when	Main commands
A — API push	You have an access token	<code>npm run agility:push / --upsert</code>
B — Excel CSV import	Token not available; use Agility's Excel import	<code>npm run agility:export</code> then paste into <code>.xls</code>

Part A — API push (recommended when you have a token)

A1. Create an Agility project (Scope)

1. In Agility, create or open a project.
2. Note the **exact Scope / project name** (spelling and spacing matter).

A2. Create an access token

1. Agility → **My Settings** → **Access Tokens**.
2. Create a token with **Product Planning** permissions.
3. Copy the token once (you will not see it again).

A3. Configure `.env.agility`

A `.env.agility` file holds Agility URL, token, and Scope. Do **not** commit it.

macOS / Linux

```
cp .env.agility.example .env.agility
```

Windows (Command Prompt or PowerShell)

```
copy .env.agility.example .env.agility
```

Edit `.env.agility` :

Variable	What to put
AGILITY_BASE_URL	Instance URL, e.g. <code>https://your-instance.com</code> (no trailing slash)
AGILITY_ACCESS_TOKEN	Token from A2
AGILITY_SCOPE	Exact project name from A1

A4. Dry-run (preview, no writes)

In the Cursor terminal (repo root):

```
npm run agility:push:dry-run
```

Optional — one feature only:

```
npm run agility:push:dry-run -- --feature 1
```

Confirm epic/story/test payloads look right. Fix Scope or token errors before a real push.

A5. Verify connection (optional)

```
npm run agility:verify
```

A6. First push — create backlog

Full push (all discovered features):

```
npm run agility:push
```

Or one feature:

```
npm run agility:push -- --feature 1
```

With an explicit project name (overrides `.env.agility` Scope if needed):

```
npm run agility:push -- --project "Your Exact Project Name"
npm run agility:push -- --feature 2 --project "Your Exact Project Name"
```

Mode	Command	Behavior
Full	<code>npm run agility:push</code>	Create epics + stories + tests for all features
Feature	<code>npm run agility:push -- --feature N</code>	Feature N only; creates epic if missing
Upsert	<code>npm run agility:push -- --feature N -- upsert</code>	Update existing by Reference/Name; create missing (epic must already exist)

Open Agility → Product Planning and confirm epics, stories, and tests appear.

A7. Re-sync after a spec change

When you edit user stories or Gherkin in `features/feature-N-....md` :

```
npm run agility:push -- --feature N --upsert
```

Do not re-run a plain create push for the same items — that **duplicates** assets. Prefer `--upsert` for updates.

Part B — Excel CSV import (no API token)

Use this if your course uses Agility's Excel import instead of the Bulk API.

B1. Create or pick Agility project

Same as A1 — note the **exact** Scope name.

B2. Generate CSVs

From the repo root (replace with your Scope name):

```
npm run agility:export
# or with an explicit name:
node scripts/export-agility-import.mjs --project "Your Exact Project Name"
```

Default `npm run agility:export` uses project `"0C CS Speckit"` — change the command if your Scope differs.

Output (repo root unless the script documents otherwise):

File	Paste into Agility template tab
PortfolioItem.csv	Portfolio Item
Story.csv	Story
Test.csv	Test

B3. Download Agility Excel template

From Agility, download the [Basic or Advanced import template](#) (`.xls`).

B4. Fill the workbook

1. Open the template in Excel.
2. Copy **values** from each CSV into the matching worksheet (headers must align).
3. Required story columns include **AssetType, Name, Scope**.
4. Save as `.xls` .

If import fails on Status ("Multiple StoryStatus..."), leave **Status** blank — see [Agility import docs](#).

B5. Import in Agility

1. Product Planning → **Import**.
2. Upload the `.xls` .
3. Review backlog: epics → stories → tests.
4. Assign stories to **sprints / timeboxes** in Agility (not in the Markdown).

B6. Re-export after spec changes

```
node scripts/export-agility-import.mjs --project "Your Exact Project Name"
```

Excel import **creates new items only** — it does not update existing ones. Re-importing the same file **duplicates**. Import only net-new items, or switch to API `--upsert` (Part A) for updates.

Part C — When to sync (process)

Per [framework.md — Merge checklist + Agility sync](#):

Change	Action
New feature file <code>features/feature-N-*.md</code>	Push/export that feature (create epic + stories + tests)
Edited stories or Gherkin on feature N	<code>agility:push -- --feature N --upsert</code> (or careful new-only Excel import)

Implementation-only (no story/AC text change)	No Agility re-sync required
Sprint planning	Assign iterations in Agility

Specs stay authoritative. Agility mirrors planning structure for the team.

Part D — Checkpoint questions

1. What becomes an Epic, a Story, and a Test in Agility?
2. Why must `AGILITY_SCOPE / --project` match the Agility project name exactly?
3. Why is `--upsert` safer than a second full create push after editing Gherkin?
4. Where are sprints assigned — in `features/` or in Agility?
5. Should you commit `.env.agility` ? Why or why not?

Part E — Deliverables

1. **Canvas:** Submit your **GitHub repository link** (if required) and/or a short note with your Agility project name.
2. **Demo:** Show the professor or GA your Agility backlog (at least one epic with stories and acceptance tests linked from a feature spec), and explain whether you used **API push** or **Excel import**.

Evidence	Expected
Agility project	Epics for features; stories with References; tests under stories
Local config (API path)	<code>.env.agility</code> present locally, not committed
Spec link	Story description or custom field points at <code>features/feature-N-...md</code>

Quick command cheat sheet

```
# API path
cp .env.agility.example .env.agility # Windows: copy .env.agility.example .env.agility
# edit AGILITY_BASE_URL, AGILITY_ACCESS_TOKEN, AGILITY_SCOPE

npm run agility:push:dry-run
npm run agility:verify
npm run agility:push
npm run agility:push -- --feature 3
npm run agility:push -- --feature 3 --upsert

# Excel path
npm run agility:export
# or:
node scripts/export-agility-import.mjs --project "Your Exact Project Name"
# → PortfolioItem.csv, Story.csv, Test.csv → paste into Agility .xls → Import
```

Troubleshooting

Problem	Fix
Auth / 401 on push	Check token; recreate Access Token; confirm AGILITY_BASE_URL
Scope not found	AGILITY_SCOPE / --project must match Agility project name exactly
Duplicate epics/stories	You re-ran create; use --upsert for updates; clean duplicates in Agility
Excel Status errors	Leave Status blank on import
Empty export	Ensure features/feature-N-*.md files exist with # Feature: ... and stories/Gherkin
Token in git	Remove from history; rotate token; keep .env.agility gitignored

Related

- Full reference: [docs/agility-import/README.md](#)
- Agility: [Access Token Authentication](#) · [Excel import](#)